ChatGPT said:

React Hooks – Complete Interview Questions and Answers

For React interviews, especially fresher and internship roles, prepare these questions in order: basic Hooks, `useState`, `useEffect`, `useRef`, `useContext`, `useReducer`, and custom Hooks.

1. React Hooks basics

Q1. What are Hooks in React?

Hooks are special functions that let functional components use React features such as state, context, refs, and effects.

Q2. Why were Hooks introduced?

Hooks allow us to reuse stateful logic and use React features in functional components without writing class components.

Q3. What are the main React Hooks?

- `useState()`
- `useEffect()`
- `useContext()`
- `useRef()`
- `useReducer()`
- `useMemo()`
- `useCallback()`
- `useLayoutEffect()`
- `useId()`
- `useTransition()`
- `useDeferredValue()`
- `useSyncExternalStore()`
- `useImperativeHandle()`
- `useDebugValue()`
- `useActionState()` and `useOptimistic()` for newer React features.

Q4. What are the Rules of Hooks?

1. Call Hooks at the top level of a function component or custom Hook.
2. Do not call Hooks inside loops, conditions, or nested functions.
3. Do not call Hooks from ordinary JavaScript functions or class components.

Q5. Can we create our own Hook?

Yes. A custom Hook is a JavaScript function whose name starts with `use` and which can call other Hooks.

Example: `useFetch()`, `useLocalStorage()`.

2. `useState()` questions

Q6. What is `useState()`?

`useState()` is a Hook used to store and update state in a functional component.

```javascript
const [count, setCount] = useState(0);
```

Here, `count` is the current state, `setCount` updates it, and `0` is the initial value.

Q7. How do you update state?

```javascript
setCount(count + 1);
```

Q8. What happens when state changes?

React schedules a re-render so the UI can reflect the updated state.

Q9. What is the difference between props and state?

Props	State
Passed from parent to child	Managed by the component
Read-only to the receiving component	Updated using a setter or reducer
Used to pass data	Used to store changing data

Q10. Why should we not modify state directly?

Direct mutation does not properly notify React of the change and can lead to stale or inconsistent UI. Use the state setter.

```javascript
// Wrong
count = count + 1;

// Correct
setCount(count + 1);
```

Q11. What is a functional state update?

It calculates the next state using the previous state.

```javascript
setCount(prev => prev + 1);
```

This is useful when the next value depends on the previous value.

Q12. How do you store an object in state?

```javascript
const [user, setUser] = useState({
  name: "Poli",
  age: 21
});

setUser(prev => ({ ...prev, age: 22 }));
```

The spread syntax preserves the other properties while updating `age`.

3. `useEffect()` questions

Q13. What is `useEffect()`?

`useEffect()` lets you synchronize a component with external systems, such as APIs, timers, subscriptions, and browser events.

Q14. What are the three common dependency-array patterns?

JavaScript

```
// 1. After every committed render
useEffect(() => {
  console.log("Rendered");
});

// 2. After initial mount
useEffect(() => {
  console.log("Mounted");
}, []);

// 3. After mount and when count changes
useEffect(() => {
  console.log("Count changed");
}, [count]);
```

Q15. What is a cleanup function?

A function returned by an effect that cleans up resources before the effect runs again or when the component unmounts.

JavaScript

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Running");
  }, 1000);

  return () => clearInterval(id);
}, []);
```

Q16. Can we call an API using `useEffect()`?

Yes. For example:

JavaScript

```
useEffect(() => {
  async function getUsers() {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    const data = await response.json();
    console.log(data);
```

```
  }

  getUsers();
}, []);
```

Q17. What causes an infinite loop in `useEffect()`?

An effect updates state, which causes a render, and the effect then updates that state again without a condition that stops the cycle.

Q18. Does `useEffect()` run before or after rendering?

It runs after React commits the render to the screen. `useLayoutEffect()` is used when an effect must run before the browser repaints.

4. `useRef()` questions

Q19. What is `useRef()`?

`useRef()` stores a value that persists between renders without causing a re-render when that value changes.

Q20. What are the uses of `useRef()`?

- Accessing DOM elements.
- Storing timer IDs.
- Keeping mutable values between renders.

Example:

JavaScript

```
import { useRef } from "react";

function InputExample() {
  const inputRef = useRef(null);

  return (
    <div>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>
        Focus
      </button>
    </div>
  );
}
```

Q21. Difference between `useState()` and `useRef()`?

`useState()`	`useRef()`
Stores state for rendering	Stores a persistent mutable value
Updating it schedules a re-render	Updating `.current` does not trigger a re-render
Used for UI data	Used for DOM access and non-rendering values

5. `useContext()` questions

Q22. What is `useContext()`?

It lets a component read a value provided by a React Context, avoiding the need to pass that value through every intermediate component.

Q23. What is prop drilling?

Passing props through multiple intermediate components that do not need those props themselves.

Q24. Give an example of `useContext()`.

JavaScript

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function Home() {
  const theme = useContext(ThemeContext);

  return <h1>Theme: {theme}</h1>;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Home />
    </ThemeContext.Provider>
  );
}

export default App;
```

Output: `Theme: dark`

Q25. What is the difference between props and context?

Props pass data directly from a parent to a child. Context allows multiple components to access a shared value without passing it through every level.

6. `useReducer()` questions

Q26. What is `useReducer()`?

It manages state using a reducer function that receives the current state and an action, then returns the next state.

Q27. When should we use `useReducer()` instead of `useState()`?

Use `useReducer()` when state transitions are complex, involve multiple related values, or are easier to manage with action-based logic.

Q28. Give an example.

JavaScript

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, {
    count: 0
  });

  return (
    <div>
```

Q29. What is `dispatch()`?

It sends an action to the reducer so React can calculate the next state.

7. `useMemo()` and `useCallback()` questions

Q30. What is `useMemo()`?

It caches the result of a calculation between renders until its dependencies change.

```javascript
const result = useMemo(() => {
  return numbers.reduce((sum, n) => sum + n, 0);
}, [numbers]);
```

Q31. What is `useCallback()`?

It caches a function definition between renders until its dependencies change.

```javascript
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

Q32. Difference between `useMemo()` and `useCallback()`?

`useMemo()`	`useCallback()`
Caches a calculated value	Caches a function
`useMemo(() => value, deps)`	`useCallback(() => {}, deps)`
Useful for expensive calculations	Useful when function identity matters

Q33. Should we use them everywhere?

No. They have their own overhead. Use them when caching meaningfully improves performance or prevents unnecessary work.

8. Other important Hooks

Hook	Purpose
`useLayoutEffect()`	Runs after DOM changes but before browser repaint
`useId()`	Generates IDs suitable for accessibility relationships

`useTransition()`	Marks updates as non-urgent
`useDeferredValue()`	Lets part of the UI lag behind a rapidly changing value
`useImperativeHandle()`	Customizes the ref handle exposed to a parent
`useSyncExternalStore()`	Subscribes to an external store
`useDebugValue()`	Displays helpful labels for custom Hooks in React DevTools
`useActionState()`	Manages state associated with an Action
`useOptimistic()`	Displays optimistic UI while an Action is in progress

9. Custom Hook interview questions

Q34. What is a custom Hook?

A custom Hook is a reusable function that starts with `use` and can use other Hooks to share stateful logic between components.

Example:

JavaScript

```
import { useState } from "react";

function useCounter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount(prev => prev + 1);
  }

  return { count, increment };
}

function App() {
  const { count, increment } = useCounter();

  return (
    <button onClick={increment}>
      Count: {count}
    </button>
  );
```

Q35. Do two components share the same state when they use the same custom Hook?

No. Each component gets its own Hook state unless the Hook connects to a shared store or other shared mechanism.

10. Tricky interview questions

Q36. Can we use Hooks inside class components?

No. Hooks are designed for function components and custom Hooks.

Q37. Can Hooks be called inside an `if` condition?

No. Hooks must be called at the top level, in the same order on every render.

Q38. Does changing a ref trigger a re-render?

No. Changing `ref.current` does not itself trigger a re-render.

Q39. Does every state update immediately update the variable?

No. State behaves as a snapshot for the current render. The setter schedules an update; it does not change the value in the already-running code.

Q40. What is the difference between `useEffect()` and `useLayoutEffect()`?

`useEffect()` is preferred for most side effects. `useLayoutEffect()` runs before the browser repaints and is useful for layout measurement or preventing visual flicker.

Practice these coding questions

Coding checklist 0 of 10

- [ ] 1. Create a counter using useState()
  Easy
- [ ] 2. Fetch and display API data using useEffect()
  Easy
- [ ] 3. Focus an input using useRef()
  Easy
- [ ] 4. Share a theme using useContext()
  Medium
- [ ] 5. Build a counter using useReducer()
  Medium
- [ ] 6. Create a reusable custom useCounter() Hook

Medium

- 7. Prevent unnecessary calculations using useMemo()
  Medium
- 8. Clean up a timer or event listener in useEffect()
  Medium
- 9. Explain why an effect causes an infinite loop
  Medium
- 10. Use useCallback() with a memoized child component
  Advanced

Practice next question

Placement tip: First become confident with `useState`, `useEffect`, `useRef`, and `useContext`. These are good starting points for fresher interviews, then move to `useReducer`, custom Hooks, and performance Hooks.